

Report conversation

안녕 gpt야, 내가 지금 visibility API를 사용해서 웹소켓 연결을 관리하고 있어. 앱의 시작부분에 document.addEventListener로 visibilitychange를 감지하는 핸들러 함수를 달아줬고, 웹소켓이 정상적으로 wasClean하게(또는 code=1000으로) 종료되었을 때 document.removeEventListener로 해당 핸들러 함수를 제거해주도록 코딩했어. 근데 실제 개발환경에서는 eventListener가 정상적으로 제거되지 않고 계속 살아있는거같아. 이럴 때 어떻게 제대로 이벤트리스너를 제거할 수 있을까?

안녕하세요! 웹소켓과 visibility API를 사용하는 개발 중에 발생한 문제에 대해 물어보셨군요. 이벤트 리스너가 정상적으로 제거되지 않는 문제는 종종 발생할 수 있는데, 몇 가지 접근 방법을 통해 이 문제를 해결할 수 있습니다.

- 명시적인 함수 정의 사용:** 이벤트 리스너를 추가할 때 사용한 함수를 변수에 저장한 다음, 제거할 때 그 함수를 활용하여 제거합니다. 이렇게 하면 정확한 함수를 식별하여 제거할 수 있습니다.

javascript

 Copy code

```
function handleVisibilityChange(event) {
    // 처리 로직
}

// 이벤트 리스너 추가
document.addEventListener('visibilitychange', handleVisibilityChange);

// 이벤트 리스너 제거
document.removeEventListener('visibilitychange', handleVisibilityChange);
```

- 익명 함수 사용:** 이벤트 리스너를 추가할 때 익명 함수를 사용하면, 제거할 때도 동일한 익명 함수를 전달하여 제거할 수 있습니다.

javascript

 Copy code

```
const handleVisibilityChange = function(event) {
    // 처리 로직
```

```
}

```

```
// 이벤트 리스너 추가

```

```
document.addEventListener('visibilitychange', handleVisibilityChange);

```

```
// 이벤트 리스너 제거

```

```
document.removeEventListener('visibilitychange', handleVisibilityChange);

```

3. **이벤트 객체 활용:** 이벤트 리스너 내에서 이벤트 객체를 활용하여 현재 리스너가 연결된 이벤트를 식별하고 제거할 수 있습니다.

```
javascript

```

 Copy code

```
document.addEventListener('visibilitychange', function handleVisibilityChange(event) {

```

```
    // 처리 로직

```

```
    // 이벤트 리스너 제거

```

```
    document.removeEventListener('visibilitychange', handleVisibilityChange);
});

```

위의 방법 중 하나를 선택하여 이벤트 리스너를 제거하도록 수정해보세요. 만약 이런 접근 방법을 사용해도 문제가 해결되지 않는다면, 다른 요인이 문제를 유발하고 있을 수 있습니다. 이 경우에는 추가적인 디버깅이 필요할 수 있습니다.

okay, 내 문제가 뭔지 알아냈어. 이벤트리스너를 추가하는 과정에서 함수를 아래와 같이 넘겨줘서 이름이 없는 함수였기 때문에 remove할 때도 무슨 이벤트리스너를 없애야하는지 알 수 없어 지워지지 않던거였어.

```
document.addEventListener('visibilitychange', () => {
    handleVisibilityChange(ws_url, id, navigate).catch(error =>
        console.error(error),
    );
});

```

내가 addEventListener에서 listener를 위와 같이 적은 이유는 typescript eslint 중 Promise-returning function provided to attribute where a void return was expected.라는 에러를

피하기 위함이었어. eslint 에러를 해결하면서 정상적으로 이벤
트리스너를 추가하고 삭제할 수 있게 하는 방법이 없을까?

확인해 보니, 코드를 보면 이벤트 리스너를 익명 함수로 정의하고 있으며, 해당 익명 함수 내부에서
는 이름 있는 함수 `handleVisibilityChange` 를 호출하고 있습니다. TypeScript에서는 함수의
반환 타입이 `void`가 아닌 경우, 해당 함수를 이벤트 리스너로 사용하면 "Promise-returning
function provided to attribute where a void return was expected."와 같은 에러가 발생할 수
있습니다.

이러한 상황에서 가장 깔끔한 해결 방법은 아래와 같이 함수를 분리하여 사용하는 것입니다.

javascript

 Copy code

```
// handleVisibilityChange 함수 정의
async function handleVisibilityChangeWrapper(ws_url, id, navigate) {
  try {
    await handleVisibilityChange(ws_url, id, navigate);
  } catch (error) {
    console.error(error);
  }
}

// 이벤트 리스너 추가
document.addEventListener('visibilitychange', () => {
  handleVisibilityChangeWrapper(ws_url, id, navigate);
});
```

위의 코드처럼, `handleVisibilityChangeWrapper` 함수를 정의하여 거기에서
`handleVisibilityChange` 함수를 호출하고, 오류 처리도 하도록 구성합니다. 그리고 이벤트 리
스너 내부에서는 `handleVisibilityChangeWrapper` 함수를 호출하도록 하면 됩니다. 이렇게 하
면 TypeScript의 eslint 규칙을 준수하면서도 이벤트 리스너를 추가하고 삭제할 수 있을 것입니다.

근데 `() => {`
 `handleVisibilityChangeWrapper(ws_url, id, navigate);`
`}` 이 함수도 익명함수 아니야?

맞습니다, () => { ... } 역시 익명 함수입니다. 죄송합니다, 제 설명이 혼동스러웠습니다.

TypeScript의 eslint 에러를 피하려면, 아래와 같이 함수를 정의하고 사용하는 방법을 고려해볼 수 있습니다.

javascript

 Copy code

```
// handleVisibilityChange 함수 정의
async function handleVisibilityChangeWrapper(ws_url, id, navigate) {
  try {
    await handleVisibilityChange(ws_url, id, navigate);
  } catch (error) {
    console.error(error);
  }
}

// 이벤트 리스너 함수 정의
function visibilityChangeListener() {
  handleVisibilityChangeWrapper(ws_url, id, navigate);
}

// 이벤트 리스너 추가
document.addEventListener('visibilitychange', visibilityChangeListener);

// 이벤트 리스너 제거
document.removeEventListener('visibilitychange', visibilityChangeListener);
```

위의 코드처럼, 실제 이벤트 리스너를 추가할 때는 visibilityChangeListener 함수를 사용하고, 이 함수 내에서는 handleVisibilityChangeWrapper 함수를 호출하는 방식으로 처리하면, TypeScript eslint 에러를 회피하면서도 이벤트 리스너를 추가하고 제거할 수 있습니다.

너가 알려준 코드대로 하면 불필요하게 함수를 2번 3번 감싸는 함수를 만들어야해서 보기 좋지 않은 것 같아. 이벤트리스너로 넘겨줄 함수가 인자가 필요한 함수일 때, 인자를 사용하는 함수를 명명해서 removeEventListener로 감지할 수 있게 하는 방법이 있어?

네, 인자를 사용하는 함수를 이용하여 `removeEventListener` 를 수행하는 방법이 있습니다. 이렇게 하려면 인자를 사용하는 함수를 외부 스코프에서 선언하고, 이를 이벤트 리스너 추가 및 제거에서 활용하면 됩니다. 이렇게 하면 별도의 중첩 함수를 만들 필요가 없습니다.

아래는 이와 관련된 코드 예시입니다:

javascript

 Copy code

```
// 이벤트 리스너 함수 정의
function visibilityChangeListener(ws_url, id, navigate) {
  return () => {
    handleVisibilityChange(ws_url, id, navigate).catch(error =>
      console.error(error)
    );
  };
}

// 이벤트 리스너 추가
const listener = visibilityChangeListener(ws_url, id, navigate);
document.addEventListener('visibilitychange', listener);

// 이벤트 리스너 제거
document.removeEventListener('visibilitychange', listener);
```

위의 코드에서 `visibilityChangeListener` 함수는 인자를 받아 해당 인자들을 사용하여 리스너 함수를 생성하는 역할을 합니다. 리스너 함수가 생성된 이후에는 동일한 리스너 함수를 `addEventListener` 및 `removeEventListener` 에서 활용합니다.

이 방법을 사용하면 불필요한 중첩을 피하면서도 원하는 인자를 사용하는 함수를 명명하여 이벤트 리스너를 추가하고 제거할 수 있습니다.

```
그럼 const handleVisibilityChange = async (
  ws_url: string,
  id: UserStatus['id'],
  navigate: NavigateFunction,
) => {
  if (document.hidden) {
```

```

console.log('background');
// 페이지가 백그라운드로 가려질 때 실행할 로직
} else {
  console.log('now visible');
  // 기존 소켓이 연결 상태였다가 화면 꺼짐으로 인해 끊어진 상태.
  // id값이 변하지 않았고 다시 같은 url로 소켓 연결하면 됨
  console.log(socket);
  if (!socket) {
    socket = new WebSocket(ws_url);
    console.log(
      동일한 ws_url로 웹소켓이 재연결되었습니다. 연결이
      끊어진 공백기간 동안 상태에 변화가 있는지 확인합니다.
      ${ws_url},
    );

    const response = await UserApi.postCheckStatus({
      assign_id: id });

    if ('status' in response) {
      const { status } = response;
      if (
        !window.location.pathname.includes(status) &&
        ![ 'failed', 'finish', 'cancel' ].includes(status)
      ) {
        // 현재 라우트 pathname과 실제 상태가 일치하지 않고,
        // 배정실패/하차/호출취소가 아닌 상태에서만 리다이렉션 발생
        console.log(
          서버 상태와 클라이언트 상태 간 불일치가 감지되었습니다.
          ${status} 라우트로 이동합니다.,
        );
        switchRoutesByStatus(status, navigate);
      } else {
        console.log(`${status} 상태이므로 소켓 재연결을
        시도하지 않습니다.);
      } else {
        throw new Error(response.detail);
      }
    } else console.log(소켓 연결이 살아있는 상태입니다.
    ${socket.url});
  }

```

}

};

위 함수가 현재 Promise<void> 리턴 값을 가지는데 Promise 객체 말고 void를 반환하게 하는 방법이 있어?

handleVisibilityChange 함수가 async 키워드로 비동기 함수로 선언되어 있어서 Promise 객체를 반환합니다. async 함수를 사용하면 내부에서 발생하는 Promise를 암시적으로 처리하고 결과를 Promise로 감싸서 반환하기 때문입니다. 따라서 Promise<void> 를 반환하는 것은 async 함수의 자연스러운 동작입니다.

만약 void 를 직접 반환하게 하려면, async 함수를 사용하지 않고 일반 함수로 만들어야 합니다. 그러나 await 를 사용하여 비동기 작업을 처리하는 부분에서 변경이 필요하게 될 수 있습니다. async / await 를 사용하면 비동기 작업을 보다 명확하고 동기적으로 다룰 수 있으므로, async 함수의 장점을 최대한 활용하는 것이 좋습니다.

만약 Promise<void> 를 반환하는 것이 코드 내에서 문제를 일으키는 상황이라면, 그 부분을 좀 더 자세히 알려주시면 해당 부분을 개선하는 방법을 제시해 드릴 수 있을 것입니다.

async/await를 사용하지 않고 .then()/.catch() api를 써도 promise 객체를 반환하는건 똑같지 않아? 아닌가?

네, 맞습니다. async/await 와 .then()/.catch() 모두 Promise 객체를 다루는 두 가지 다른 방식일 뿐, 내부적으로는 Promise를 반환하게 됩니다.

async/await 는 비동기 코드를 보다 동기적으로 작성할 수 있도록 하는 문법적인 도구입니다. async 함수 내에서 await 를 사용하면 비동기 작업의 결과를 기다린 후에 다음 작업을 처리할 수 있게 되며, await 표현식은 자동으로 해당 Promise가 resolve될 때까지 기다리게 됩니다.

반면에 .then() 및 .catch() 를 사용하면 비동기 작업의 결과를 콜백 함수를 통해 처리합니다. 이 역시도 Promise 객체의 상태 변화를 다루는 것으로, .then() 은 Promise가 resolve된 경우에, .catch() 는 Promise가 reject된 경우에 호출됩니다.

둘 중 어느 방식을 사용하더라도 Promise 객체의 결과를 다루며, 결국엔 비동기 작업의 결과를 처리하는 방법에 차이가 있을 뿐입니다. 코드의 가독성과 유지 보수성을 고려하여 async/await 나 .then() /. catch() 중 더 편리하다고 생각되는 방식을 사용하시면 됩니다.

handleVisiblityChange가 Promise객체를 반환하는게
eventListener를 등록하는데 있어서 걸림돌이 되고 있어.
eventListener로는 void를 반환하는 함수만 등록이 가능한데,
handleVisiblityChange 함수는 Promise 객체를 반환해서 바로
listener 함수로 등록할 수가 없어.

이해했습니다. 이러한 경우에는 아래와 같이 다른 접근 방법을 사용하여 문제를 해결할 수 있습니다.

1. **IIFE (Immediately Invoked Function Expression) 사용:** handleVisibilityChange 함수를 IIFE로 감싸서 즉시 실행하고 그 결과로 반환되는 함수를 이벤트 리스너로 등록합니다.

javascript

 Copy code

```
document.addEventListener('visibilitychange', (async () => {  
    await handleVisibilityChange(ws_url, id, navigate);  
}))());
```

2. **콜백 함수 사용:** handleVisibilityChange 함수에서 비동기 작업이 끝난 후에 실행할 콜백 함수를 전달하여, 콜백 함수 내에서 이벤트 리스너를 등록합니다.

javascript

 Copy code

```
function handleVisibilityChangeWithCallback(callback) {  
    handleVisibilityChange(ws_url, id, navigate)  
        .then(() => {  
            if (typeof callback === 'function') {  
                callback();  
            }  
        })  
        .catch(error => {  
            console.error(error);  
            if (typeof callback === 'function') {  
                callback();  
            }  
        })  
};
```

```
document.addEventListener('visibilitychange', () => {  
  handleVisibilityChangeWithCallback(() => {  
    // 여기에 다음 작업을 수행할 코드 작성  
  });  
});
```

위의 두 가지 방법은 `handleVisibilityChange` 함수의 Promise 반환을 다루는 방법을 보다 효과적으로 처리할 수 있는 방법입니다. 첫 번째 방법은 IIFE를 사용하여 함수를 감싸면서 동시에 실행하는 방식이며, 두 번째 방법은 콜백 함수를 통해 비동기 작업 완료 후 다음 작업을 수행하는 방식입니다. 선택하신 방법에 따라 코드를 조정해보시기 바랍니다.